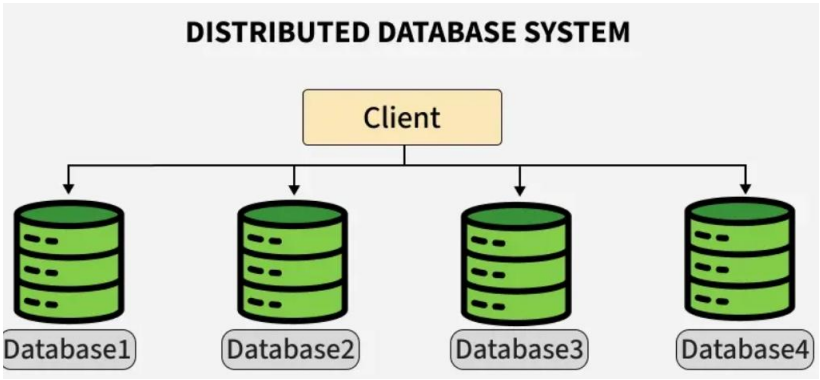
DISTRIBUTED DATABASE SYSTEM (DDBS)

A Distributed Database System (DDBS) is a collection of multiple databases spread across different physical locations, connected via a network.

A distributed system manages data across various sites while making it appear as a single database to users.

It improves data availability, reliability, and performance by enabling local access, parallel processing, and fault tolerance.



Distributed Database Types

Some of the type of distributed database system are:

1. Homogeneous Database:

In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used all are the same at all sites. Hence, they're easy to manage.

Features:

- Unified query language and interface.
- Low integration complexity.
- Efficient synchronization.

Example: A bank with branches in different cities uses Oracle DB at every location. All databases have the same structure and are synchronized regularly.

2. Heterogeneous Database

In a heterogeneous distributed database, different sites may use different DBMSs, schemas, or data models, making query processing and transactions difficult. Some sites may not even be aware of others, so translation mechanisms are needed for communication.

Features:

- Supports interoperability between diverse systems.
- Complex query optimization and transaction management.
- Useful in mergers or collaborations between organizations.

Example: A logistics company uses MySQL for inventory, MongoDB for vehicle tracking, and PostgreSQL for billing. Integration middleware allows unified querying across these platforms.

3. Client-Server Distributed Database System

In this model, the server stores and manages the database, while clients send queries over the network. It offers centralized control with distributed access, making it ideal for enterprise systems and web applications. Clients can be lightweight while the server handles heavy processing. Example: Web application interacting with a central PostgreSQL server.

Features:

- Simplifies resource management.
- Central servers can be optimized for performance.
- Easily scalable with more clients.

Example: An e-commerce website where the frontend (client) is hosted separately and interacts with a central PostgreSQL server to manage orders, users, and inventory.

4. Peer-to-Peer Distributed Database System

Here, all nodes are equal, with no fixed client or server roles. Each node can store data and also process queries, leading to decentralized control. It supports fault tolerance and high availability.

Example: Blockchain networks like Ethereum, where each node maintains a part of the distributed ledger.

Features:

- No single point of failure.
- Useful in decentralized and distributed apps.
- High availability and data redundancy.

5. Cloud-Based Distributed Database System

These systems are deployed on cloud platforms and span multiple geographic regions for scalability and reliability. They abstract infrastructure details and are offered as DBaaS, making them ideal for dynamic workloads. Example: Google Cloud Spanner and Amazon DynamoDB used for global applications.

Features:

- Automatic scaling and replication.
- Pay-as-you-use pricing.
- Global availability and disaster recovery.

Example:

Google Cloud Spanner: Global-scale relational database.

Amazon DynamoDB: Key-value and document database with high performance.

Azure Cosmos DB: Multi-model, globally distributed DBMS.

BASIC CONCEPTS

1. Replication

In replication, copies of the same data are stored at two or more sites. If every site has the full database, it's called full replication. This improves data availability and allows faster, parallel query processing. However, updates must be made at all sites, or data may become inconsistent. It also adds overhead and makes concurrency control more complex.

2. Fragmentation

In this approach, the relations are fragmented (i.e., they're divided into smaller parts) and each of the fragments is stored in different sites where they're required. It must be made sure that the fragments are such that they can be used to reconstruct the original relation (i.e, there isn't any loss of data).

Fragmentation is advantageous as it doesn't create copies of data, consistency is not a problem. Fragmentation of relations can be done in two ways:

- **Horizontal fragmentation - Splitting by rows**
  The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.
- **Vertical fragmentation - Splitting by columns**
  The schema of the relation is divided into smaller schemas. Each fragment must contain a common candidate key so as to ensure a lossless join.

In certain cases, an approach that is hybrid of fragmentation and replication is used.

3. Concurrency Control

Concurrency control ensures data remains accurate when multiple transactions run at the same time. Without it, issues like lost updates or dirty reads can occur. Its goal is to make parallel transactions behave as if run one by one. Common methods include locking, timestamps, and optimistic concurrency.

4.Semantic Heterogeneity

Semantic heterogeneity happens when different databases use the same data labels but with different meanings, formats, or units. For example, one system may store salary in dollars, another in rupees. This can cause confusion during data integration, so resolving it is important for accurate results.

CHARACTERISTICS OF DISTRIBUTED DATABASES

1. Location Independence:

Data is spread across multiple sites, but the system manages it as one unified database. Users and applications do not need to know the physical location of the data. This is also known as Distribution Transparency, because the DDBMS hides all details about where the data actually resides.

2. Scalability:

Distributed databases can grow horizontally by adding more nodes, servers, or storage locations. As data volume and user traffic increase, new sites can be added without disrupting the existing system. This makes distributed systems ideal for large or expanding organizations.

3. Performance:

Workload is distributed across multiple sites, which:

- Reduces processing load on any single node
- Allows queries to run in parallel
- Provides faster execution

4. Data Locality:

Data is stored closer to the users or applications that frequently access it.

5. Local Autonomy:

Each site or node in the distributed system can manage its own data and operations independently. Local DBMSs can:

- Process local queries
- Maintain local security policies
- Operate even when disconnected from other nodes

6. Distributed Query Processing:

Queries may involve data stored at multiple sites. The DDBMS optimizes query execution by choosing the most efficient way to:

- Retrieve data
- Minimize communication cost
- Reduce overall processing time
- This makes distributed queries seamless to users.

7. Fault Tolerance:

Besides replication, distributed systems use recovery methods so that:

- Failed sites can be restored
- The system continues running without major disruption
- Fault tolerance ensures reliability in real-world environments.

DESIGN ISSUES IN DDBMS

Designing a Distributed Database System (DDBS) is significantly more complex than designing a centralized one. The core challenge is balancing the trade-offs between **performance** (speed), **reliability** (availability), and **consistency** (data accuracy).

1. Data Distribution

The first major decision is how the data should be distributed across the different sites of the network. Designers must determine:

- Whether to store the data centrally, partially, or fully distributed
- How to place data so that access is fast and communication cost is low

Proper distribution ensures better performance and efficient resource usage.

2. Data Fragmentation

Fragmentation involves dividing a database table into smaller, more manageable pieces called fragments.
Types include:

- Horizontal fragmentation (by rows)
- Vertical fragmentation (by columns)
- Hybrid fragmentation (combination)

The main issue is choosing a fragmentation strategy that allows efficient data access while still enabling the reconstruction of the original table when needed.

3. Data Replication

Replication involves keeping multiple copies of data at different sites to improve availability and reliability.
Key considerations include:

- How many replicas to maintain
- Where to place them
- How to keep all copies synchronized

While replication improves fault tolerance, it also increases the complexity of update management.

4. Distributed Query Processing

Queries in a distributed system may involve data stored at multiple locations. Major issues include:

- Choosing the most efficient execution plan
- Minimizing data transfer between sites
- Using parallel processing where possible

The objective is to ensure fast query execution despite the data being distributed.

5. Distributed Transaction Management

Transactions must maintain ACID properties even when they span multiple sites. Key problems to address:

- Ensuring atomicity (all-or-nothing execution)
- Coordinating commits across sites (e.g., Two-Phase Commit protocol)
- Handling failures during distributed transactions

This is one of the most complex aspects of distributed database design.

6. Concurrency Control

When multiple users access or modify distributed data simultaneously, conflicts can occur. Designers must select suitable methods such as:

- Distributed locking
- Timestamp ordering
- Optimistic concurrency control

The aim is to maintain consistency without causing delays or deadlocks.

7. Fault Tolerance and Recovery

Distributed systems are more prone to failures such as site crashes or network breakdowns. Design issues include:

- How to recover lost data
- How to restore failed sites
- How to maintain system consistency during failures

Fault tolerance ensures that the system continues operating even when components fail.

DATA FRAGMENTATION

The process of dividing the database into smaller multiple parts or sub-tables is called fragmentation. The smaller parts or sub-tables are called fragments. Each fragment is a logical unit of data and are stored at different locations.

Data fragmentation should be done in a way that the reconstruction of the original parent database from the fragments is possible. The restoration can be done using UNION or JOIN operations.

Fragments are independent (no fragment can be derived from another), and users don't need to know that the data is fragmented. This is known as ***fragmentation transparency***.

Database fragmentation is of three types: Horizontal fragmentation, Vertical fragmentation, and Mixed or Hybrid fragmentation.

1. HORIZONTAL FRAGMENTATION

Horizontal fragmentation splits a table by rows based on conditions on one or more attributes. Each fragment contains a subset of rows, which are then stored at different sites. This helps in localizing data access.

In relational algebra, it's represented using the SELECT (σ) operation on the table.

$\sigma_p(R)$

Where:

- σ is relational algebra operator for selection
- p is the condition satisfied by a horizontal fragment

Note that a union operation can be performed on the fragments to construct Relation R. Such a fragment containing all the rows of Relation R is called a complete horizontal fragment.

For example, consider an EMPLOYEE table (T) :

Eno	Ename	Design	Salary	Dep
101	A	abc	3000	1
102	B	abc	4000	1
103	C	abc	5500	2
104	D	abc	5000	2
105	E	abc	2000	2

This EMPLOYEE table can be divided into different fragments like:

- EMP 1 = $\sigma_{Dep = 1}$ EMPLOYEE
- EMP 2 = $\sigma_{Dep = 2}$ EMPLOYEE

These two fragments are:

1. T1 fragment of Dep = 1

Eno	Ename	Design	Salary	Dep
101	A	abc	3000	1
102	B	abc	4000	1

2. T2 fragment of Dep = 2

Eno	Ename	Design	Salary	Dep
103	C	abc	5500	2
104	D	abc	5000	2
105	E	abc	2000	2

Now, here it is possible to get back T as **T = T1 ∪ T2 ∪ .... ∪ TN**

2. VERTICAL FRAGMENTATION

Vertical fragmentation splits a table by columns (attributes), storing different columns on different sites. Since each site may not need all columns, this improves efficiency. To reconstruct the original table, each fragment must include a common key (like the primary key or a tuple ID) so that rows can be joined correctly using a natural JOIN.

$\Pi\ a_1, a_2,..., a_n\ (T)$

- π is relational algebra operator
- a1...., an are the attributes of T
- T is the table (relation)

For example, for the EMPLOYEE table we have T1 as :

Eno	Ename	Design	Tuple_id
101	A	abc	1
102	B	abc	2
103	C	abc	3
104	D	abc	4
105	E	abc	5

For the second sub table of relation after vertical fragmentation is given as follows :

Salary	Dep	Tuple_id
3000	1	1
4000	2	2
5500	3	3
5000	1	4
2000	4	5

This is T2 and to get back to the original T, we join these two fragments T1 and T2 as $\pi_{EMPLOYEE}$ **(T1 ⋈ T2)**

3. MIXED FRAGMENTATION

Hybrid (or mixed) fragmentation combines vertical and horizontal fragmentation-first splitting a table by columns, then dividing those fragments by rows (or vice versa). It uses both SELECT (σ) and PROJECT (π) operations. This is useful when simple horizontal or vertical fragmentation isn't sufficient for data distribution.

Mixed fragmentation can be done in two different ways:

1. The first method is to first create a set or group of horizontal fragments and then create vertical fragments from one or more of the horizontal fragments.
2. The second method is to first create a set or group of vertical fragments and then create horizontal fragments from one or more of the vertical fragments.

The original relation can be obtained by the combination of JOIN and UNION operations which is given as follows:

$\sigma_P(\pi\ a_1,a_2..,a_n(T))$
$\pi\ a_1,a_2....,a_n\ (\sigma_p(T))$

DATA REPLICATION IN DDBMS

Data Replication in DBMS refers to the process of storing copies of the same data at multiple sites or nodes within a distributed database system. The main purpose of replication is to improve data availability, reliability, performance and fault tolerance.

Goal of Data Replication:

- Increase the availability of data
- Speed up the query evaluation
- To improve reliability and fault tolerance in distributed systems.

Types:

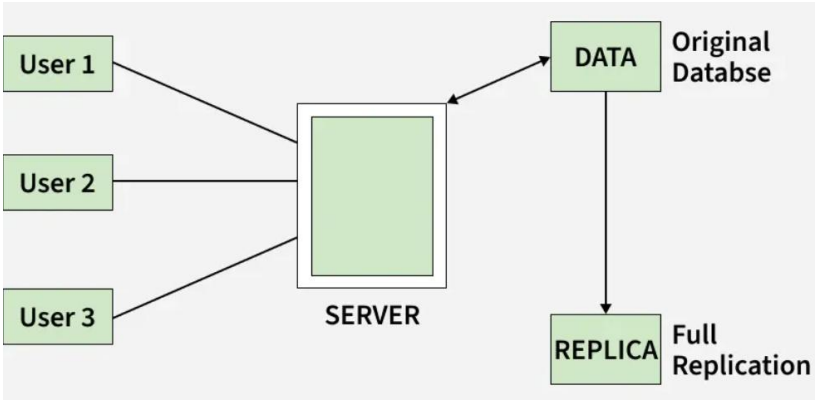
Synchronous Replication: The replica will be modified immediately after some changes are made in the relation table. So, there is no difference between original data and replica.

Asynchronous Replication: The replica will be modified after commit is fired on DB.

REPLICATION SCHEMES

1. Full Replication

In full replication, the entire database is replicated at every site. Offers maximum availability and faster local access. However, it increases storage requirements and update complexity.



Advantages:

- High data availability
- Faster query execution (local access)
- Improved performance for global queries

Disadvantages:

- Difficult concurrency management
- Slow update propagation
- High storage and network costs
- Complex consistency maintenance

2. No Replication

In no replication, each data item is stored at only one location in the distributed system.

Advantages:

- Reduced concurrency issues
- Simplified recovery and management

Disadvantages:

- Poor data availability (single point of failure)
- Slower query response due to centralized access

3. Partial Replication

In partial replication, only selected fragments of the database are replicated based on their importance or access frequency. Balances between storage efficiency and availability.

Advantages:

- Optimized storage
- Flexibility based on data importance
- Better performance for critical data

Disadvantages:

- Complex management and configuration
- Possible inconsistent data access across sites

ADVANTAGES OF DATA REPLICATION

- Higher data availability
- Improved system reliability
- Faster response time
- Reduced network traffic
- Better read performance
- Increased fault tolerance

DISADVANTAGES OF DATA REPLICATION

- High update cost
- Increased storage requirement
- Complex consistency control
- Requires efficient synchronization techniques

DISTRIBUTED TRANSPARENCY:

Distribution Transparency means that the system hides all the complexities of data distribution from the user, allowing them to interact with the database as if it were stored in one single location.

This is one of the most important goals of a DDBMS because it makes the system easy to use, flexible, and independent of physical storage details.

TYPES:

1. Location Transparency

Location transparency refers to the ability to access distributed resources without knowing their physical or network locations. It hides the details of where resources are located, providing a uniform interface for accessing them.

Importance: Enhances system flexibility and scalability by allowing resources to be relocated or replicated without affecting applications.

Examples: DNS (Domain Name System): Maps domain names to IP addresses, providing location transparency for web services.

2. Replication Transparency

Users do not need to know whether data is replicated, how many copies exist, or how updates are propagated across replicas.

The DDBMS automatically:

- Chooses the nearest or most efficient replica
- Ensures all copies remain consistent
- Manages update synchronization

Importance:

- Improves performance by using local replicas
- Ensures availability even if one site fails

3. Fragmentation Transparency

Fragmentation transparency ensures that users are not aware whether a table is **fragmented**, how many fragments exist, or where those fragments are stored. This enables users to query upon any table as if it were un-fragmented. Thus, it hides the fact that the table the user is querying on is actually a fragment or union of some fragments.

The DDBMS automatically:

- Identifies the right fragments
- Fetches data from all sites
- Combines results and returns them to the user

PARALLEL DATABASES

Parallel databases use multiple processors working together to improve performance, speed, throughput, and scalability when executing database operations.

They do this by dividing a big task into smaller sub-tasks that run simultaneously.

A parallel DBMS is a DBMS that runs across multiple processors or CPUs and is mainly designed to execute query operations in parallel, wherever possible. The parallel DBMS link a number of smaller machines to achieve the same throughput as expected from a single large machine.

There are three architectural designs for parallel DBMS. They are as follows:

1. Shared Memory Architecture
2. Shared Disk Architecture
3. Shared Nothing Architecture

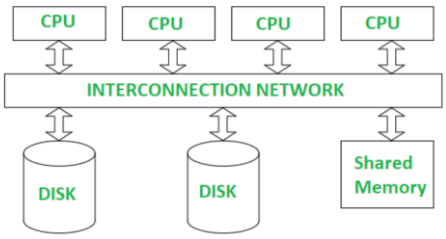
1. SHARED MEMORY ARCHITECTURE (SMA)

In the Shared Memory Architecture, multiple CPUs are connected through an interconnection network and all of them share the same main memory and disk storage.

A single copy of a multithreaded operating system and DBMS can run across these CPUs because they access the same memory space.

This type of system is called Symmetric Multiprocessing (SMP), where several processors work together and share memory equally.

Shared memory systems can range from small workstations with a few processors to larger servers.



Advantages:

- It has high-speed data access for a limited number of processors.
- The communication is efficient.

Disadvantages:

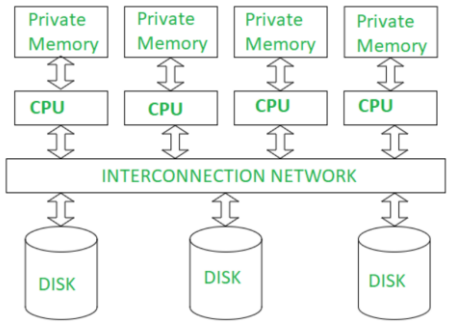
- It cannot use beyond 80 or 100 CPUs in parallel.
- The bus or the interconnection network gets block due to the increment of the large number of CPUs.

2. SHARED DISK ARCHITECTURE (SDA)

In the Shared Disk Architecture, multiple CPUs are connected through an interconnection network. Each CPU has its own private memory, but all CPUs share the same disk storage.

Because the memory is not shared, every node runs its own copy of the operating system and DBMS. This architecture is considered loosely coupled and works well for applications that need a centralized data store.

Shared disk systems are often called **clusters**.



Advantages:

- The interconnection network is no longer a bottleneck each CPU has its own memory.
- Load-balancing is easier in shared disk architecture.
- There is better fault tolerance.

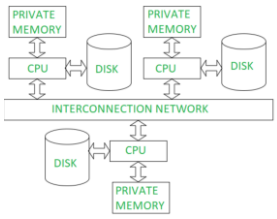
Disadvantages:

- If the number of CPUs increases, the problems of interference and memory contentions also increase.
- There's also exists a scalability problem.

3. SHARED NOTHING ARCHITECTURE (SNA)

In the Shared Nothing Architecture, each processor has its own private memory and its own disk storage.
The processors (or nodes) are connected through a high-speed interconnection network, but they do not share any memory or disk with each other.

Since no two CPUs can access the same disk or memory, each node works independently. This architecture is also called **Massively Parallel Processing (MPP)** because many processors can be added to scale the system.



Advantages:

- It has better scalability as no sharing of resources is done
- Multiple CPUs can be added

Disadvantages:

- The cost of communications is higher as it involves sending of data and software interaction at both ends

PARALLEL QUERY PROCESSING

Parallelism in a query allows us to parallel execution of multiple queries by decomposing them into the parts that work in parallel. This can be achieved by shared-nothing architecture.

Parallelism is also used in fastening the process of a query execution as more and more resources like processors and disks are provided.

We can achieve parallelism in a query by the following methods:

1. I/O parallelism
2. Intra-query parallelism
3. Inter-query parallelism
4. Intra-operation parallelism
5. Inter-operation parallelism

Types of Parallelism

There are two main ways to parallelize a query:

1. Inter-Query Parallelism

The system executes multiple different queries from different users at the same time.

Goal: Throughput. It ensures the system can handle many concurrent users without queuing.

Mechanism: If User A asks for "Sales Data" and User B asks for "HR Data," the system routes these to different nodes so they don't block each other.

2. Intra-Query Parallelism

Breaking a single complex query into sub-tasks and running them simultaneously. This is the core of distributed processing.

Sub-types:

- Intra-Operation Parallelism (Partitioned): The same task (e.g., a huge sort or search) is performed on different subsets of data at the same time.

Example: If a table is horizontally fragmented across 4 nodes, all 4 nodes search their local fragment for "Customer ID 101" simultaneously.

- Inter-Operation Parallelism (Pipelined): Different tasks within the same query are executed concurrently.
  Example: While Node A is retrieving data, Node B is already sorting the data that Node A has finished sending. This "assembly line" approach hides latency.

QUERY OPTIMIZATION IN DISTRIBUTED DATABASES

Query optimization is the process of finding the most efficient execution plan for a query by minimizing:

- Communication cost (most expensive in DDBMS)
- Data transfer
- Disk I/O
- CPU time
- Total response time

Distributed query optimization is more complex because data is stored at different sites.

Factors Considered in Distributed Query Optimization

A. Data Distribution: Data may be fragmented, replicated, or partitioned across multiple sites. Optimizer chooses plans that minimize movement of data between sites.

B. Communication Cost: Transferring data between sites is expensive; the optimizer tries to reduce:

- number of messages
- size of data being transferred

C. Local Processing Cost: Operations performed at each site should be efficient and balanced.

D. Parallel Execution Opportunities: Optimizer identifies parts of the query that can run in parallel.

OPTIMIZATION TECHNIQUES

A. Global Query Optimization: Creates a distributed execution plan using information about data location, fragmentation, and network costs.

Chooses the best site to perform joins and aggregations.

B. Semi-Join Strategy: Used to reduce data transfer during joins.

Only necessary attributes or join keys are sent across sites, not full tables.

C. Join Location Optimization: Decides where join should happen:

- move small table to the site of the big table
- move fragments to a central site
- perform partial joins at each site

D. Query Decomposition: Breaks a global query into smaller sub-queries processed at different sites, often in parallel.

E. Cost-Based Optimization: The optimizer evaluates many possible execution plans and selects the one with the lowest estimated cost.

OBJECT-BASED DATABASES (OODBMS)

An Object-Based Database, often referred as Object-Oriented Database Management System (OODBMS), is a database system where information is represented in the form of objects as used in object-oriented programming (OOP).1

Unlike relational databases (RDBMS) which think in terms of tables and rows, OODBMS thinks in terms of objects, classes, and inheritance. This aligns the database structure directly with the application code (like Java, C++, or Python).

MOTIVATION: WHY DO WE NEED THEM?

Object-Based Databases were created because traditional relational databases could not handle the needs of modern applications that use complex data.

Impedance Mismatch: In programs, data is stored as objects (like a Car object containing Engine and Wheel objects).
But relational databases store everything in flat tables, so the objects must be broken into tables and then rebuilt again using joins.
This constant conversion is slow and needs extra mapping code.
An OODBMS stores the object directly, so no translation is needed.

Complex Data Types: Relational databases struggle with unstructured or complex data like multimedia (video/audio), computer-aided design (CAD) drawings, or geospatial data. OODBMS handles these naturally as objects.

Performance: In applications with complex relationships (like a Bill of Materials or a social network), OODBMS is faster because it follows direct object links instead of using slow JOIN operations.

Version Management: Engineering and design applications often need to maintain multiple versions of the same object (e.g., "Design v1.0" vs "Design v1.1"). OODBMS often supports this natively.

KEY FEATURES

Object-Based Databases borrow their core concepts directly from Object-Oriented Programming.

Object Identity (OID): Every object has a unique, immutable identifier (OID) independent of its data values. Even if you change every property of an object, it remains the same object. In contrast, RDBMS identifies rows by Primary Keys (values).

Encapsulation: Data (attributes) and behavior (methods) are bundled together. You interact with the data through defined methods, ensuring data integrity.

Inheritance: You can create a hierarchy of classes. A SportsCar class can inherit attributes from a generic Car class. This promotes code and schema reusability.

Polymorphism: The ability for different objects to respond to the same message (method call) in different ways. For example, a draw() method works differently for a Circle object than a Square object.

Complex Objects: Objects can be nested. An object can contain other objects (composition/aggregation), allowing for deep, hierarchical structures.

ARCHITECTURE

The architecture of an OODBMS is designed to support the storage and retrieval of objects seamlessly.

The architecture generally consists of the following layers:

Object Manager: The core component. It handles the interface between the application and the physical storage. It manages object identities, transaction management, and concurrency control.

Schema Manager: Stores the meta-data (class definitions, inheritance hierarchies, and method signatures).

Query Processor: Optimizes and executes queries. Unlike SQL, queries here often involve traversing object paths (e.g., Employee.Department.Address.City).

Storage Manager: Manages the physical disk space. It is responsible for efficient allocation (clustering related objects together on disk to improve fetch speed) and indexing.

Note: Many OODBMS architectures are "Client-Server."21 The Client (application) often performs much of the processing (method execution), while the Server acts more as a sophisticated object storage page server. This is different from RDBMS, where the server does almost all the processing.

Summary: OODBMS vs. RDBMS

Feature	Relational Database (RDBMS)	Object-Based Database (OODBMS)
Data Structure	Tables, Rows, Columns	Objects, Classes
Relationships	Foreign Keys, Joins	Object References (Pointers)
Identification	Primary Key (Value-based)	Object ID (System-generated)
Access Language	SQL (Standardized)	OQL (Object Query Language) or Native API
Best Use Case	Accounting, Banking, Transactional data	CAD, Simulation, Real-time Systems, Complex Data

OBJECT-ORIENTED DATA MODEL (OODM)

The Object-Oriented Data Model (OODM) is the theoretical framework that structures data around "objects" rather than "records" or "tables." It extends the principles of Object-Oriented Programming (OOP) into database design.

The Object-Oriented Model in DBMS or OODM is the data model where data is stored in the form of objects. This model is used to represent real-world entities.

The data and data relationship are stored together in a single entity known as an object in the Object Oriented Model. The Object-Oriented Database Management System is built on top of Object Oriented Model.

We can use the Object Oriented Model in DBMS to store real-world entities. Here, we can store pictures, audio, video, and other types of data, which was previously impossible to store with the

relational approach (Even though we can store video and audio in the relational database, it is generally not recommended).

Here are the core concepts that define this model:

1. The Object and Object Identity (OID)

In the OODM, any real-world entity is represented as an object. Unlike relational databases where a row is identified by a primary key (like EmployeeID = 101), an object in an OODM is identified by a unique Object Identifier (OID).

- **Immutable**: The OID is generated by the system and never changes, even if the object's properties (like name or address) change.
- **Invisible**: The OID is generally used by the system to manage pointers and references, not by the user directly.

2. Attributes and Methods

An object is not just a container for data; it is a package containing both:

- **Attributes**: The values describing the object (e.g., Color, Weight, Price). Attributes can be:
  - Simple: Primitives like integers or strings.
  - Complex: References to other objects
- **Behavior (Methods):** The code or operations that can be performed on the object.

3. Classes

The class acts as a blueprint or schema. The class defines the structure (attributes) and behavior (methods).An individual object is an instance of a class.

4. Inheritance (The "IS-A" Relationship)

This is a powerful feature that allows new classes to be derived from existing classes. The derived class (Subclass) inherits attributes and methods from the parent class (Superclass).

Benefit: This reduces redundancy and makes schema updates easier.

5. Complex Objects (Aggregation / Composition)

OODM supports storing nested or combined objects, allowing real-world structures to be represented naturally.

- **Composition**: An object is made up of other objects. Example: A Car object contains Engine, Wheel, and Chassis objects.

6. Encapsulation

Encapsulation ensures that an object's internal data is protected.

- The data inside an object cannot be accessed directly.
- Users interact with the object only through its public methods.

Benefit: Helps maintain data integrity.

7. Polymorphism: Polymorphism allows different classes to respond to the same method call in their own specific ways.

OBJECT-RELATIONAL DATABASE (ORD)

An Object-Relational Database (ORD) is a database management system (DBMS) that integrates object-oriented database model features into relational databases. ORDs aim to bridge the gap between relational databases and the object-oriented modeling techniques that are commonly used in programming languages.

This type of database supports data types, structures, and behaviors directly in the database schema and query language.

Architecture of Object-Relational Databases

ORD architecture extends traditional relational database architecture with several key enhancements:

- **Type System:** Supports user-defined types and inheritance in database schemas.
- **Table Inheritance**: Allows table definitions to inherit from other tables.
- **Complex Data Types:** Facilitates complex data types like arrays, structs, and even custom-defined types.
- **Methods:** Similar to object-oriented programming, methods (functions) can be defined on data types and stored in the database.

How Object-Relational Databases Work

Object-relational databases work by:

- **Storing Data:** Data is stored in tables with the flexibility of object-oriented features such as inheritance and polymorphism.
- **Querying Data:** SQL queries are enhanced to support complex types and object-oriented constructs, enabling more sophisticated data retrieval.
- **Handling Transactions:** Provides full ACID (Atomicity, Consistency, Isolation, Durability) properties for handling transactions, ensuring data integrity and consistency.

Benefits of Using Object-Relational Databases

- **Improved Data Integrity:** By encapsulating behaviors with data, data handling becomes more robust and consistent.
- **Increased Scalability and Flexibility:** Supports complex applications and data types without sacrificing performance.

Considerations and Challenges

- **Complexity**: Increased complexity in design and query language can lead to steeper learning curves.
- **Performance Overhead**: Additional features such as inheritance and user-defined types might introduce performance overhead.
- **Integration**: Integrating with other systems that do not support object-relational features can be challenging.

Applications of Object-Relational Databases

- Computer-Aided Design (CAD)
- Multimedia Databases
- Scientific Databases
- E-commerce Systems

DIFFERENCE BETWEEN RDBMS VS. OODBMS VS. ORDBMS.

Feature	RDBMS	OODBMS	ORDBMS
Data Model	Tabular (rows & columns)	Objects (OOP-based)	Hybrid: Tables + Objects
Representation of Data	Relational tables	Objects with attributes & methods	Tables with support for complex data types
Identity	Primary key (user-defined)	OID (system-generated)	Primary key + object identifiers for complex types
Complex Data Handling	Weak (needs joins, normalization)	Strong (supports nested & multimedia objects)	Moderate to strong (UDTs, arrays, multimedia types)
Relationship Handling	Foreign keys + JOINs	Object references (pointers)	Both joins and object references
Query Language	SQL	OQL or language bindings	Extended SQL (SQL:1999 and later)
Performance	Best for structured data & transactions	Best for object traversal & complex relationships	Balanced performance for both structured & complex data
Schema Evolution	Less flexible	More flexible (supports inheritance)	Flexible (supports object extensions)
Inheritance Support	Not supported	Fully supported	Partially supported (via UDTs)
Use Cases	Banking, ERP, HR systems, traditional applications	CAD/CAM, AI, simulations, multimedia applications	GIS, multimedia, scientific data, modern enterprise apps
Storage Method	Tables only	Direct object storage	Tables with ability to store objects or references
Integration with OOP	Weak (requires ORM tools)	Strong (native object mapping)	Moderate (object extensions inside relational model)
Learning Curve	Low	Higher (requires OOP knowledge)	Moderate

CONCURRENCY ISSUES IN DBMS

When multiple transactions run at the same time, they may interfere with each other. If proper control is not used, this leads to incorrect results. Two common problems are Lost Updates and Dirty Reads.

The database transaction consist of two major operations "Read" and "Write". It is very important to manage these operations in the concurrent execution of the transactions in order to maintain the consistency of the data.

DIRTY READ PROBLEM (WRITE-READ CONFLICT)

A dirty read problem occurs when one transaction updates an item but due to some unconditional events that transaction fails but before the transaction performs rollback, some other transaction reads the updated value. Thus creates an inconsistency in the database.

Dirty read problem comes under the scenario of Write-Read conflict between the transactions in the database.

The lost update problem can be illustrated with the scenario between two transactions T1 and T2.

- Transaction T1 modifies a database record without committing changes.
- T2 reads the uncommitted data changed by T1
- T1 performs rollback.
- T2 has already read the uncommitted data of T1 which is no longer valid, thus creating inconsistency in the database.

LOST UPDATE PROBLEMS (W - W CONFLICT)

Two transactions read the same data, both modify it, and then write it back, causing the second transaction to overwrite the first one's update, making the first update disappear.

Example:

- T1 reads the value of an item from the database.
- T2 starts and reads the same database item.
- T1 updates the value of that data and performs a commit.
- T2 updates the same data item based on its initial read and performs commit.
- This results in the modification of T1 gets lost by the T2's write which causes a lost update problem in the database.

DATABASE RECOVERY

Database recovery is the process of restoring a database to a correct, consistent, and usable state after a failure. Its primary goal is to ensure the ACID properties (specifically Atomicity and Durability) are preserved—ensuring that completed transactions persist and incomplete ones are wiped out.

CAUSES OF DATABASE FAILURE

Failures generally fall into three categories:

1. Transaction Failures

These occur within the individual software process but do not damage the storage hardware.

- **Logical Errors:** The transaction cannot complete because of some internal code error (e.g., bad input, data not found, overflow, or division by zero).
- **System Errors:** The database system itself ends the transaction (e.g., the DBMS terminates a transaction to break a deadlock).

2. System Crashes (Soft Failures)

This happens when the hardware or software freezes, causing the system to stop processing.

- Examples: Power failure, operating system crash, or RAM failure.
- Impact: Content in Volatile Memory (RAM) is lost (including buffers and logs not yet written to disk). Content on the Non-Volatile Storage (Hard Disk) remains safe.

3. Disk/Media Failures (Hard Failures)

These are the most dangerous failures where the actual storage device is damaged.

- Examples: Head crashes, disk controller failure, or file system corruption.
- Impact: Data on the hard disk is lost or corrupted. Recovery requires restoring from archival backups (tape/cloud) rather than just logs.

DATABASE RECOVERY APPROACHES

Database recovery techniques determine how transaction updates are handled, and how the system recovers after a failure using the log.

The two most common approaches are:

- Deferred Update (NO-UNDO / REDO)
- Immediate Update (UNDO/REDO)

1. Deferred Update

Also called NO-UNDO / REDO technique.

- In this technique, the database is not updated immediately.
- Only log file is updated on each transaction.
- When the transaction reaches to its commit point, then only the database is physically updated from the log file.
- In this technique, if a transaction fails before reaching to its commit point, it will not have changed database anyway. Hence there is no need for the UNDO operation. The REDO operation is required to record the operations from log file to physical database. Hence deferred database modification technique is also called as NO UNDO/REDO algorithm.

Advantages

- Simple; no undo operations
- Ensures high atomicity

Disadvantages

- High delay because updates happen only at commit
- Requires enough memory/buffer to store all updates

2. Immediate Update

Also called UNDO/REDO technique.

In this technique, the database is updated during the execution of transaction even before it reaches to its commit point.

If the transaction gets failed before it reaches to its commit point, then a ROLLBACK Operation needs to be done to bring the database to its earlier consistent state. That means the effect of operations need

to be undone on the database. For that purpose both Redo and Undo operations are both required during the recovery. This technique is known as UNDO/REDO technique.

Advantages

- Faster because updates happen progressively
- Suitable for systems requiring high performance
- Supports long-running transactions

Disadvantages

- More complex because both undo and redo are required
- Requires strict Write-Ahead Logging (WAL)

Difference between Deferred update and Immediate update:

Deferred Update	**Immediate Update**
In a deferred update, the changes are not applied immediately to the database.	In an immediate update, the changes are applied directly to the database.
The log file contains all the changes that are to be applied to the database.	The log file contains both old as well as new values.
In this method once rollback is done all the records of log file are discarded and no changes are applied to the database.	In this method once rollback is done the old values are restored to the database using the records of the log file.
Concepts of buffering and caching are used in deferred update method.	Concept of shadow paging is used in immediate update method.
The major disadvantage of this method is that it requires a lot of time for recovery in case of system failure.	The major disadvantage of this method is that there are frequent I/O operations while the transaction is active.
In this method of recovery, firstly the changes carried out by a transaction on the data are done in the log file and then applied to the database on commit. Here, the maintained record gets discarded on rollback and thus, not applied to the database.	In this method of recovery, the database gets directly updated after the changes made by the transaction and the log file keeps the old and new values. In the case of rollback, these records are used to restore old values.

INTRODUCTION OF SHADOW PAGING

Shadow paging is a recovery technique used in DBMS to ensure that transactions are atomic and durable. Instead of keeping detailed logs like log-based recovery, shadow paging maintains two versions of the database's page table:

1. Shadow Page Table

- Represents the old, stable state of the database (before the transaction begins).
- It never changes during the transaction.
- If a failure occurs, the system can immediately revert to this version.

2. Current Page Table

- A copy of the shadow page table made when a transaction starts.
- All updates during the transaction are made here.
- Only modified pages are replaced with new pages—this is known as copy-on-write or out-of-place updating.

HOW IT WORKS

Start Transaction: A new "current page table" is created, and its contents are copied from the previous "shadow page table," which represents the last consistent state of the database.

During Transaction: All modifications are applied to new "current pages." The original "shadow pages" remain untouched, and the "shadow page table" continues to point to them.

Commit: If the transaction completes successfully, the "current page table" is made permanent by replacing the "shadow page table". The new pages are written to disk, and the old pages are no longer needed.

Rollback (Failure): If the system crashes, the "current page table" and all changes are simply discarded. The database reverts to the state defined by the "shadow page table," ensuring atomicity and durability.

Advantages of Shadow Paging

- Fewer Disk Accesses: Requires fewer disk operations to perform tasks.
- Fast Recovery: Crash recovery is quick and inexpensive, with no need for Undo or Redo operations.
- Improved Fault Tolerance: Transactions are isolated, so a failure in one transaction does not affect others.
- Increased Concurrency: Changes are made to shadow copies, allowing multiple transactions to run simultaneously without interference.

Disadvantages of Shadow Paging

- High Commit Overhead: A large number of pages need to be flushed during the commit process, increasing the overhead.
- Data Fragmentation: Related pages may become separated, leading to fragmentation.
- Limited Concurrency: Extending the algorithm to support concurrent transactions is challenging.

INTRODUCTION TO LOGGING AND CHECKPOINTING TECHNIQUES IN DBMS

Database Management Systems (DBMS) must ensure that data remains consistent, durable, and recoverable, even after failures such as system crashes, power loss, or transaction errors. Two essential techniques used for this purpose are:

- Logging
- Checkpointing

These techniques work together under the Recovery Manager of DBMS to restore the database to a correct state.

1. LOGGING IN DBMS

Logging is the process of recording all important actions performed by transactions in a special file called the Log File (or Transaction Log).
This log acts like a black box of the database — it helps rebuild or undo changes during recovery.

WHAT IS A LOG?

A log is a sequential record stored on stable storage (disk) that stores information such as:

- Transaction start/end
- Before image (old value before update)
- After image (new value after update)
- Commit or abort markers

Format of Log Records

A typical log entry looks like:

```
<Ti, X, old_value, new_value>
```

Where:

- **T_i** = Transaction ID
- **X** = Data item
- **old_value** = value before update
- **new_value** = value after update

Why Logging is Important?

Logging supports two main recovery operations:

1) Undo (Rollback)

- Uses old values stored in the log.
- Required when a transaction fails before commit.

2) Redo

- Uses new values stored in the log.
- Required to reapply committed transactions after a crash.

Thus, logging ensures Atomicity and Durability (ACID properties).

2. CHECKPOINTING IN DBMS

Checkpointing is a periodic operation in which the DBMS:

- Saves all modified data (dirty pages) from memory to disk
- Writes a checkpoint record into the log

This creates a known consistent point in the log from which recovery can start.

Purpose of Checkpointing

- Reduces recovery time
- Avoids scanning the entire log from the beginning
- Ensures that most committed updates are already written to disk

How a Checkpoint Works?

During a checkpoint:

1. The DBMS temporarily pauses new transactions.
2. Writes all in-memory updates to disk.
3. Writes a log entry such as:

```
<START CHECKPOINT>
<END CHECKPOINT>
```

4. Resumes normal operations.

This acts as a **recovery boundary**.

3. WHY LOGGING AND CHECKPOINTING ARE USED TOGETHER?

When a crash occurs:

- Log provides detailed steps to redo committed transactions and undo incomplete ones.
- Checkpoint reduces how far back the system needs to look in the log.

Example: Without checkpoints → Recovery may need to read millions of log records. With checkpoints → Only records after the last checkpoint are scanned.

4. BENEFITS OF LOGGING AND CHECKPOINTING

Feature	Logging	Checkpointing
Purpose	Records all changes for recovery	Reduces recovery time
Helps in	Undo + Redo	Faster recovery
Stores	Transaction start/end, before/after images	Save consistent database state
Works during	Throughout transaction execution	Periodically
Recovery Scope	Complete replay or rollback	Start from last checkpoint